МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное

учреждение высшего образования

«Ярославский государственный университет имени П. Г. Демидова»

Кафедра компьютерных сетей

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Разработка интеллектуального чат-бота для автоматизации онбординга сотрудников на основе RAG-архитектуры с мультимодальным вводом

по направлению

01.03.02 Прикладная математика и информатика

Научный руководитель

к. ф.-м. н., доцент

_____И.В. Парамонов

«____» _____ 2026 г.

Студент группы ИВТ-41БО

Б. С. Атрошенко

«____» _____ 2026 г.

Ярославль, 2026

Оглавление

Аннотация 4
Введение 5
Глава 1. Постановка задачи 7
Описание предметной области 7
Цель работы и ожидаемые результаты 7
Сценарии использования системы 8
Функциональные требования 9
Нефункциональные требования 9
Задачи разработки 10
Глава 2. Используемые технологии и инструменты 11
2.1. Принцип работы RAG-систем 11
2.2. Оркестрация LLM-пайплайнов: LangChain и LangGraph 12
2.3. Векторная база данных Qdrant 12
2.4. Локальный вывод языковой модели: Ollama 13
2.5. Брокер сообщений RabbitMQ 13
2.6. Redis: кэш, хранилище сессий и шина данных 14
2.7. Автоматическое распознавание речи: faster-whisper 14
2.8. Серверная и клиентская части: FastAPI и Gradio 15
Глава 3. Разработка RAG-pipeline 16
3.1. Архитектура пайплайна загрузки и индексации документов 16
3.2. Стратегии сегментации текста: сравнительный анализ 17
3.3. Реализованная схема чанкирования 18
3.4. Построение векторного хранилища 18
3.5. Алгоритм семантического поиска 18
Глава 4. Интеллектуальный агент на основе LangGraph 21
4.1. Архитектура агента 21
4.2. Описание графа состояний 21
4.3. Узел классификации намерений 21
4.4. RAG-узел: ответы по базе знаний 22
4.5. Узел обработки обычный запросов 22
4.6. Управление диалоговой памятью 23
Глава 5. Асинхронная обработка задач с использованием RabbitMQ 25
5.1. Обоснование применения брокера сообщений 25
5.2. Топология обмена сообщениями 25
5.3. Публикатор задач 26
5.4. Потребитель сообщений и обработчики задач 27
Глава 6. Потоковая передача ответов: Redis Pub | Sub и SSE 255
6.1. Архитектурное решение потоковой передачи 285
6.2. Реализация Redis Pub/Sub для доставки токенов 28
6.3. Server-Sent Events в FastAPI 28
Глава 7. Мультимодальный ввод: Обработка голосовых запросов 30
7.1. Обоснование выбора ASR-движка 30
7.2. Реализация модуля транскрибации 30
7.3. Интеграция распознавания речи в общий пайплайн 31

Глава 8. Пользовательский интерфейс и управление базой знаний 32
8.1. Интерфейс чата 32
8.2. Модуль управления документами 32
8.3. REST API администрирования 32
Глава 9. Конфигурировани и развёртывание системы 34
9.1. Управление конфигурацией через Pydantic Settings 34
9.2. Жизненный цикл FastAPI-приложения 34
9.3. Контейнеризация с Docker Compose 34
Заключение 36
Список литературы 38

Аннотация

Выпускная квалификационная работа посвящена проектированию и реализации интеллектуальной системы поддержки онбординга сотрудников на основе технологии Retrieval-Augmented Generation (RAG). Система принимает запросы в текстовом и голосовом форматах, классифицирует намерение пользователя, выполняет семантический поиск по корпоративной базе знаний и доставляет сформированный ответ в режиме потоковой передачи токенов.

В работе реализованы: RAG-пайплайн полного цикла с векторным хранилищем Qdrant; агент на основе LangGraph с классификацией запросов; асинхронная обработка задач через брокер сообщений RabbitMQ; потоковая доставка ответов по схеме Redis Pub/Sub → Server-Sent Events; модуль распознавания речи на базе faster-whisper; веб-интерфейс Gradio и административный REST API на FastAPI.

Все инфраструктурные компоненты развёртываются через Docker Compose. Языковая модель, модель эмбеддингов и Automatic Speech Recognition (ASR) работают полностью локально, без обращений к внешним облачным сервисам.

Объём работы: 9 глав, 15 источников, 4 таблицы.

Введение

Процесс адаптации новых сотрудников в компании — одна из ключевых задач управления человеческими ресурсами. Современные компании располагают обширными корпоративными базами знаний, включающими регламенты, политики, инструкции по внутренним инструментам и описания бизнес-процессов, однако разрозненность этих документов существенно затрудняет эффективное использование накопленного опыта.

Появление языковых моделей (Large Language Models, LLM) открыло возможности для создания интеллектуальных ассистентов, способных предоставлять ответы на вопросы о корпоративных процессах в режиме естественного диалога. Однако применение LLM без дополнительных механизмов привязки к конкретной базе знаний сопряжено с проблемой «галлюцинаций» — генерацией фактически неверных, но правдоподобно звучащих утверждений. Технология Retrieval-Augmented Generation (RAG), впервые формализованная Lewis et al. [1], решает эту проблему: при обработке каждого запроса система извлекает из документального хранилища наиболее релевантные фрагменты и передаёт их языковой модели в качестве контекста.

Работа посвящена проектированию и реализации системы поддержки сотрудников, получившей рабочее название FinBridge RAG Assistant. Разработанная система принимает запросы в текстовом и голосовом форматах, классифицирует намерение запроса, извлекает релевантные фрагменты из корпоративной базы знаний, формирует ответ с помощью локально развёрнутой модели и доставляет его пользователю в режиме потоковой передачи токенов, создавая эффект «живого» печатающего ответа.

Одним из ключевых архитектурных решений системы является применение асинхронного брокера сообщений RabbitMQ, позволяющего немедленно возвращать управление клиенту при получении запроса, обрабатывая вычислительноёмкие операции в фоне. Для доставки потока токенов от фонового воркера к браузеру применяется связка Redis Pub/Sub и Server-Sent Events (SSE). Персистентная память диалога поддерживается через Redis с применением прогрессивной суммаризации.

Актуальность работы определяется совокупностью практических и технических факторов. Задача автоматизации онбординга востребована широким кругом организаций, не располагающих бюджетом на приобретение чат-ботов уровня enterprise. Реализованная архитектура целиком построена на открытом программном обеспечении: языковая модель, векторная база данных, модель распознавания речи — всё запускается локально и не требует лицензионных затрат.

Практическим результатом работы является полностью функционирующая система, развёрнутая в Docker-окружении, с веб-интерфейсом на основе Gradio и REST API на FastAPI. Функциональность продемонстрирована на синтетически подготовленном наборе корпоративных документов компании FinBridge в формате Markdown.

Глава 1. Постановка задачи

Описание предметной области

Онбординг представляет собой организованный процесс введения нового сотрудника в рабочий контекст компании: знакомство с корпоративной культурой, регламентами, внутренними инструментами, командными процессами и бизнес-направлениями. В крупных организациях накопленные знания распределены по десяткам документов, вики-страниц и обучающих материалов. Для нового сотрудника ориентация в этом массиве информации требует значительных временны́х затрат и неизбежно порождает поток вопросов к коллегам.

Типичные проблемы традиционного онбординга включают следующие. Во-первых, сотрудники вынуждены прерывать рабочий процесс коллег рутинными вопросами, ответы на которые зафиксированы в документации, но труднодоступны для быстрого поиска. Во-вторых, корпоративный поиск по вики-системам требует точного знания терминологии и местонахождения документа — навыки, которых у новичка нет. В-третьих, качество передаваемых знаний неравномерно: каждый наставник расставляет акценты по-своему, что приводит к противоречивым представлениям о регламентах. Наконец, голосовая коммуникация всё шире используется в удалённых командах, однако корпоративные базы знаний голосовой ввод не поддерживают.

Интеллектуальный чат-бот с RAG-архитектурой устраняет описанные проблемы системным образом: система доступна круглосуточно, предоставляет единообразные ответы с указанием источника, принимает как текстовые, так и голосовые запросы, и адаптирует ответы к контексту текущего диалога благодаря механизму памяти сессии.

Предметная область применения системы — финансовая компания условного профиля FinBridge. Для демонстрации функциональности был сформирован набор синтетических корпоративных документов в формате Markdown, охватывающий типичные темы онбординга: политику безопасности, процедуры согласования, описание внутренних инструментов и кадровые регламенты.

Цель работы и ожидаемые результаты

Целью настоящей работы является спроектировать и реализовать интеллектуальную систему для поддержки онбординга сотрудников, которая принимает запросы в текстовом и голосовом форматах и формирует точные ответы по корпоративной базе знаний с использованием RAG-архитектуры, а затем доставляет их пользователю в режиме потоковой передачи токенов.

Ожидаемые результаты работы:

1. Реализованный RAG-пайплайн полного цикла: загрузка документов → сегментация → вычисление эмбеддингов → индексация в векторной базе данных → семантический поиск → генерация ответа.
2. Интеллектуальный агент на LangGraph с классификацией намерений и двумя специализированными режимами обработки запросов.
3. Асинхронная архитектура обработки задач на основе брокера сообщений RabbitMQ, обеспечивающая немедленный отклик API.
4. Механизм потоковой доставки ответов через Redis Pub/Sub и Server-Sent Events.
5. Модуль мультимодального ввода с автоматическим распознаванием речи.
6. Веб-интерфейс пользователя и административный модуль управления базой знаний.
7. Среда развёртывания на Docker Compose с полной инфраструктурой.

Сценарии использования системы

Система предусматривает две роли пользователей: сотрудник — конечный пользователь чат-бота — и администратор базы знаний — специалист, ответственный за актуальность документов.

Роль: сотрудник

Сценарий Use Case - 1 — Текстовый запрос к базе знаний. При первой загрузке страницы система автоматически генерирует уникальный идентификатор сессии. Сотрудник вводит вопрос и нажимает «Отправить»; система регистрирует задачу и немедленно возвращает task_id. Интерфейс устанавливает SSE-соединение и начинает получать токены ответа. По завершении генерации к ответу прикрепляются ссылки на документы-источники. История диалога сохраняется в Redis.

Сценарий Use Case - 2 — Голосовой запрос. Сотрудник нажимает «Записать» и задаёт вопрос голосом; запись автоматически останавливается при завершении речи. Система транскрибирует запись и отображает расшифровку перед началом генерации ответа. Дальнейший поток идентичен Use Case - 1.

Сценарий Use Case - 3 — обычная беседа. Если запрос не связан с корпоративной базой знаний, система генерирует ответ напрямую через языковую модель без обращения к векторному хранилищу.

Роль: администратор базы знаний

Сценарий Use Case - 4 — Загрузка документа. Администратор выбирает файл Markdown; система проверяет расширение и сохраняет его в директорию документов на сервере.

Сценарий Use Case - 5 — Индексация документа. Администратор выбирает документы в таблице и нажимает «Индексировать»; система сегментирует тексты, вычисляет эмбеддинги и загружает чанки в Qdrant.

Сценарий Use Case - 6 — Удаление документа. Система удаляет соответствующие векторы из Qdrant и файлы с диска, возвращая статистику удалённых записей.

Сценарий Use Case - 7 — Переиндексация документа. При обновлении содержимого документа администратор откатывает индексацию, а затем повторно индексирует документ.

Функциональные требования

На основании анализа предметной области и сценариев использования сформулированы следующие функциональные требования к системе.

- ФТ-1. Система должна принимать текстовые запросы через HTTP POST-эндпоинт и возвращать task_id немедленно, без ожидания завершения генерации ответа.
- ФТ-2. Система должна принимать аудиофайлы с голосовыми запросами через отдельный HTTP POST-эндпоинт и автоматически их транскрибировать.
- ФТ-3. Система должна классифицировать каждый входящий запрос как knowledge_base или small_talk.
- ФТ-4. При обработке запроса к базе знаний система должна извлекать семантически близкие фрагменты документов и использовать их как контекст при генерации ответа.
- ФТ-5. Система должна доставлять ответ пользователю в режиме потоковой передачи токенов через SSE-соединение.
- ФТ-6. При обработке голосового запроса система должна опубликовать транскрипцию как отдельное SSE-событие до начала потоковой передачи токенов.
- ФТ-7. Система должна сохранять историю диалога в рамках сессии и учитывать не менее последних 6 сообщений при формировании запроса к языковой модели.
- ФТ-8. Система должна поддерживать загрузку документов Markdown через веб-интерфейс администратора.
- ФТ-9. Система должна обеспечивать индексацию, переиндексацию и удаление документов из векторного хранилища.
- ФТ-10. Ответ к запросу из базы знаний должен содержать ссылки на документы-источники.

Нефункциональные требования

- НФТ-1. Отзывчивость API. Эндпоинты регистрации задач должны возвращать ответ немедленно, не ожидая завершения обработки.

- НФТ-2. Неблокирующий событийный цикл. Синхронные CPU-bound операции (транскрибация речи) должны выполняться в пуле потоков, не блокируя asyncio event loop.
- НФТ-3. Изолированность сессий. Данные диалога каждого пользователя хранятся в изолированном пространстве ключей Redis с TTL не менее 1 часа.
- НФТ-4. Локальность вычислений. Языковая модель, модель эмбеддингов и модель ASR работают полностью локально, без обращений к внешним платным API.
- НФТ-5. Контейнеризация. Все инфраструктурные компоненты запускаются единой командой docker compose up -d.
- НФТ-6. Расширяемость базы знаний. Добавление новых документов выполняется без перезапуска приложения.
- НФТ-7. Документация API. Для всех HTTP-эндпоинтов автоматически генерируется Swagger UI.

Задачи разработки

Для достижения поставленной цели необходимо выполнить следующее:

1. Разработать RAG-пайплайн: реализовать загрузку, сегментацию документов и их индексацию в векторную базу данных Qdrant с поддержкой семантического поиска.
2. Реализовать интеллектуальный агент на основе LangGraph с классификацией намерений и двумя режимами обработки запросов (RAG и обычная беседа), а также с механизмом персистентной памяти диалога.
3. Реализовать асинхронную систему обработки задач с использованием брокера сообщений RabbitMQ, обеспечивающую немедленный отклик HTTP API.
4. Реализовать механизм потоковой доставки токенов от фонового воркера к клиентскому браузеру на основе Redis Pub/Sub и Server-Sent Events.
5. Интегрировать модуль автоматического распознавания речи на базе faster-whisper с корректным переносом синхронных операций в поток исполнения.
6. Разработать пользовательский веб-интерфейс на Gradio и административный модуль управления базой знаний с соответствующим REST API на FastAPI.

Глава 2. Используемые технологии и инструменты

2.1. Принцип работы RAG-систем

Retrieval-Augmented Generation (RAG) — архитектурный подход к созданию систем генерации текста, в котором запрос пользователя обогащается динамически извлечёнными фрагментами внешней базы знаний перед передачей языковой модели. Метод был впервые формализован в работе Lewis et al. [1] и в настоящее время является стандартом для построения корпоративных вопросно-ответных систем.

Архитектура RAG-системы включает две независимые фазы:

- Фаза индексации выполняется однократно или при обновлении документации: исходные тексты загружаются, разбиваются на фрагменты (чанки), для каждого вычисляется векторное представление (эмбеддинг), и полученные векторы вместе с текстом сохраняются в базе данных векторов.
- Фаза запроса выполняется при каждом обращении пользователя: вопрос преобразуется в вектор, выполняется поиск ближайших соседей, найденные фрагменты объединяются с вопросом в единый промпт и передаются языковой модели.

Ключевым преимуществом RAG является заземлённость ответов: языковая модель получает явные цитаты из корпоративных документов и опирается на них при генерации, что кардинально снижает частоту галлюцинаций. Дополнительным достоинством является динамическая актуализация базы знаний: переобучение модели не требуется — достаточно переиндексировать обновлённые документы. В настоящей работе три роли RAG-системы выполняют: BAAI/bge-small-en (модель эмбеддингов), Qdrant (векторная БД) и phi3:mini через Ollama (языковая модель).

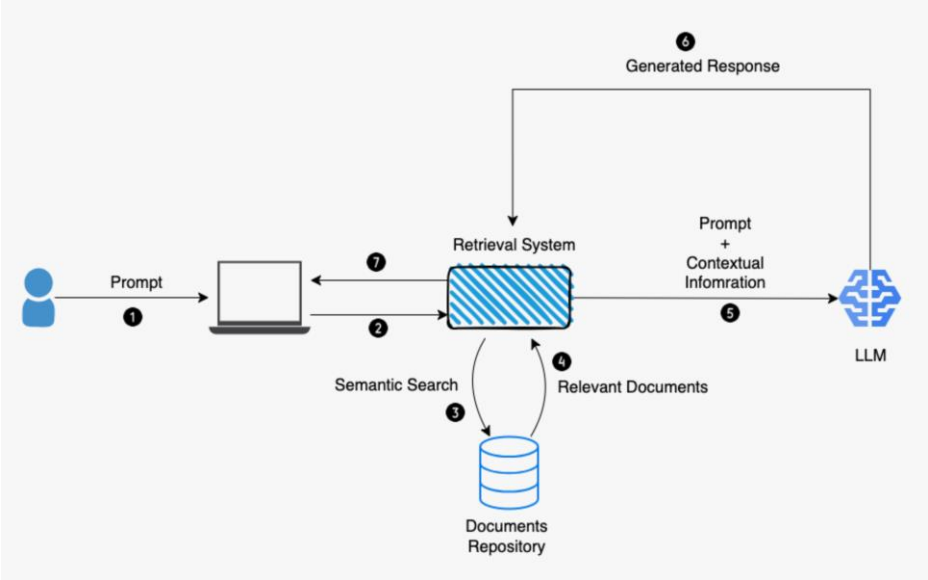


Рис. 1 - Упрощённое устройство RAG-систем

2.2. Оркестрация LLM-пайплайнов: LangChain и LangGraph

LangChain [2] — открытая библиотека Python для построения LLM-приложений, предоставляющая унифицированные абстракции для работы с языковыми моделями, шаблонами промптов, цепочками вызовов и векторными хранилищами. Центральной абстракцией является интерфейс Runnable, позволяющий компоновать компоненты в конвейеры с помощью оператора | (LangChain Expression Language, LCEL).

LangGraph [3] — расширение LangChain для построения агентов в форме ориентированных графов состояний. В отличие от линейных LCEL-цепочек, LangGraph поддерживает ветвление, условные переходы и сохранение состояния между шагами. Агент описывается как StateGraph: разработчик определяет схему состояния (Pydantic-модель), набор узлов (Python-функции или корутины) и условные рёбра между ними.

Существенным преимуществом LangGraph для потоковых сценариев является метод astream_events: он позволяет подписаться на низкоуровневые события выполнения графа, включая on_chat_model_stream (токен от LLM) и on_chain_end (завершение узла), что даёт возможность публиковать каждый токен в Redis по мере его генерации. В качестве альтернативы рассматривался AutoGen от Microsoft, однако он ориентирован на многоагентные сценарии и скрывает детали управления потоком, что неприемлемо для задачи с двусторонним ветвлением.

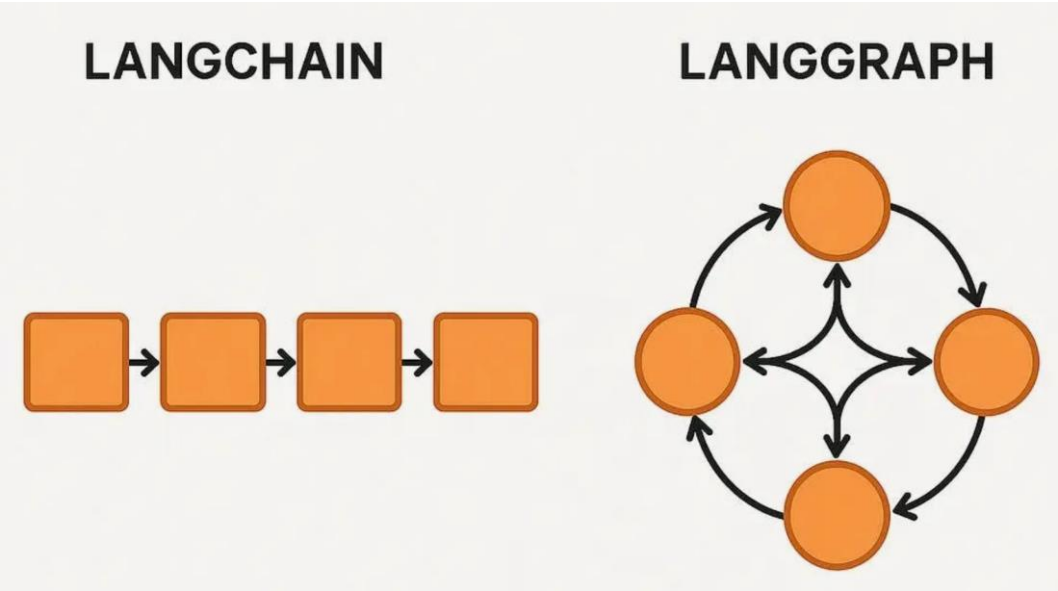


Рис. 2 - Сравнение LangChain и LangGraph

2.3. Векторная база данных Qdrant

Qdrant [4] — специализированная база данных для хранения и поиска по высокоразмерным векторам, написанная на языке Rust. Система реализует алгоритм приближённого поиска HNSW (Hierarchical Navigable Small World) и поддерживает косинусную, евклидову метрики и скалярное произведение. Qdrant предоставляет мощный

API скалярной фильтрации по произвольным полям полезной нагрузки (payload), что критично для адресного удаления чанков конкретного документа.

По сравнению с альтернативами: Pinecone требует облачного аккаунта; ChromaDB изначально проектировался как встраиваемая база без полноценного client-server режима; Weaviate имеет более высокий overhead при развёртывании в Docker. Qdrant является полностью открытым, поддерживает персистентность на диске и используется через официальный Python-клиент qdrant-client и высокоуровневую обёртку QdrantVectorStore из langchain-qdrant.

2.4. Локальный вывод языковой модели: Ollama

Ollama [5] — платформа для локального запуска языковых моделей, предоставляющая унифицированный REST API, совместимый с OpenAI Chat Completions. Ollama берёт на себя загрузку весов модели в формате GGUF (Geroge Georgiev Machine Learning), управление квантизацией и диспетчеризацию между CPU и GPU. В работе применяется модель phi3:mini — компактная квантизованная модель с 3.8 млрд параметров, функционирующая на CPU без требований к GPU.

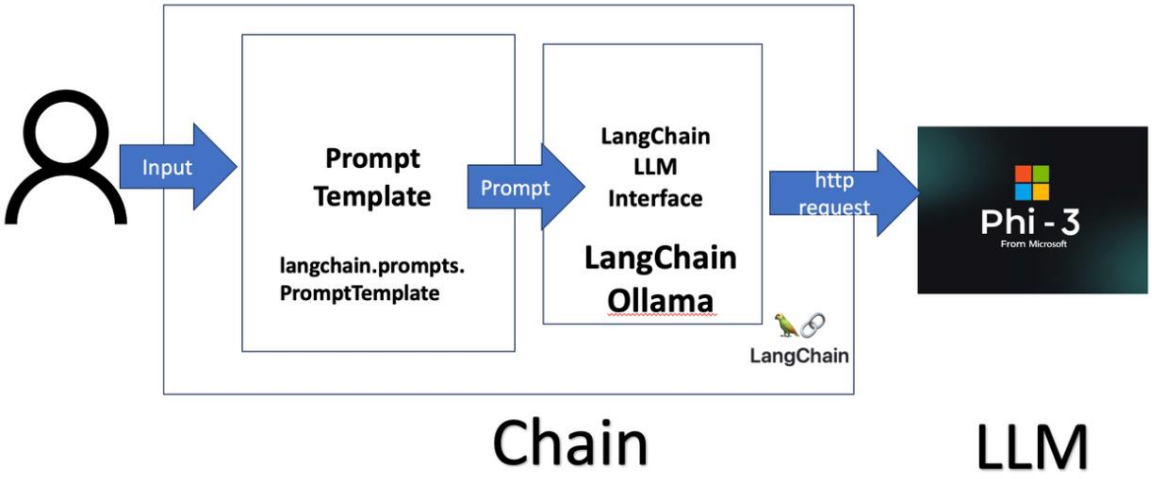


Рис. 3 - Ollama в связке с LangChain

Применение локально работающей модели обусловлено требованием конфиденциальности: корпоративные документы не должны покидать периметр инфраструктуры. Это также исключает операционные затраты на токены и обеспечивает работоспособность системы без подключения к интернету.

2.5. Брокер сообщений RabbitMQ

RabbitMQ [6] — брокер сообщений с открытым исходным кодом, реализующий протокол AMQP. Взаимодействие организовано по модели «exchange — очередь — consumer»: публикатор направляет сообщение в точку обмена с указанием ключа маршрутизации; exchange доставляет его в очередь согласно правилам связки;

потребитель извлекает сообщения и отправляет подтверждение ACK после успешной обработки.

Использование брокера сообщений решает проблему длительного HTTP-запроса: генерация ответа LLM занимает от нескольких секунд до нескольких десятков секунд, ожидание в HTTP-соединении неприемлемо. Разделение фаз «принятие задачи» (мгновенный POST) и «доставка результата» (асинхронный SSE) устраняет эту проблему принципиально. При выборе RabbitMQ с aio-pika признан оптимальным: нативная asyncio-интеграция, гибкая маршрутизация и удобный веб-интерфейс управления.

2.6. Redis: кэш, хранилище сессий и шина данных

Redis [7] — высокопроизводительное хранилище данных в оперативной памяти. В архитектуре системы Redis выполняет три независимые роли:

- Первая — хранилище истории диалога: RedisChatMessageHistory из langchain-community сериализует объекты HumanMessage и AIMessage в Redis-список под ключом chat/{session_id}.
- Вторая — хранилище прогрессивных суммаризаций: после каждого хода вычисляется краткое резюме диалога и сохраняется в Redis под ключом summary/{session_id}, предотвращая переполнение контекстного окна LLM.

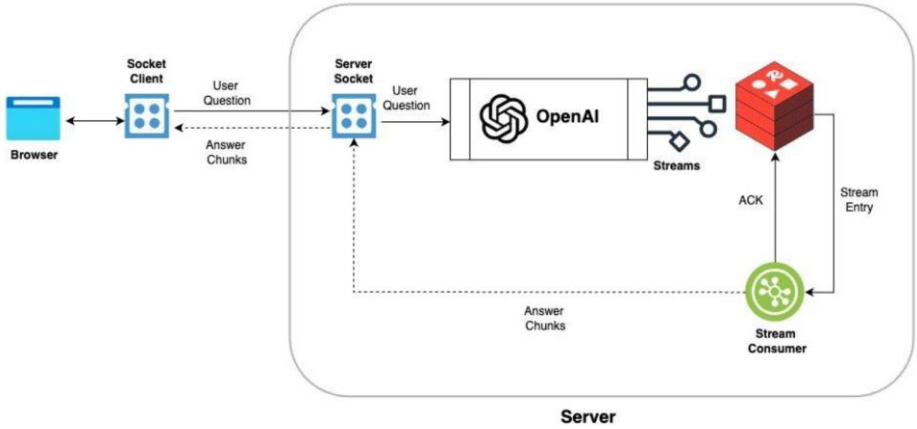


Рис. 4 - Использование Redis Pub | Sub каналов

- Третья — канал Pub/Sub для передачи токенов: воркер публикует токены в канал llm:result:{task_id}, а SSE-эндпоинт подписывается на него и транслирует события браузеру.

2.7. Автоматическое распознавание речи: faster-whisper

Faster-whisper [8] — оптимизированная реализация архитектуры Whisper от OpenAI, использующая CTranslate2 для ускоренного вывода с квантизацией int8. По сравнению с оригинальным openai-whisper на PyTorch, faster-whisper демонстрирует в 2–4

раза более высокую скорость транскрибации на CPU при аналогичном качестве. Модель поддерживает автоматическое определение языка среди 99 языков. В работе по умолчанию применяется модель base (74 МБ), обеспечивающая баланс между скоростью (2–5 секунд на вопрос длиной 10–15 секунд) и точностью.

2.8. Серверная и клиентская части: FastAPI и Gradio

FastAPI [9] — современный асинхронный веб-фреймворк Python, построенный на Starlette и Pydantic. Обеспечивает автоматическую валидацию данных, генерацию Swagger UI и нативную поддержку StreamingResponse для Server-Sent Events. Асинхронная модель позволяет обслуживать тысячи конкурентных SSE-соединений без отдельного потока на каждое соединение.

Gradio [10] — библиотека Python для создания веб-интерфейсов ML-систем. Предоставляет компоненты gr.Chatbot, gr.Audio, gr.DataFrame с минимальным объёмом кода. Для потоковых обновлений Gradio поддерживает генераторные обработчики: функция-обработчик может yield промежуточные состояния компонента. Двухвкладочный интерфейс системы реализован в двух файлах без использования HTML/CSS/JavaScript.

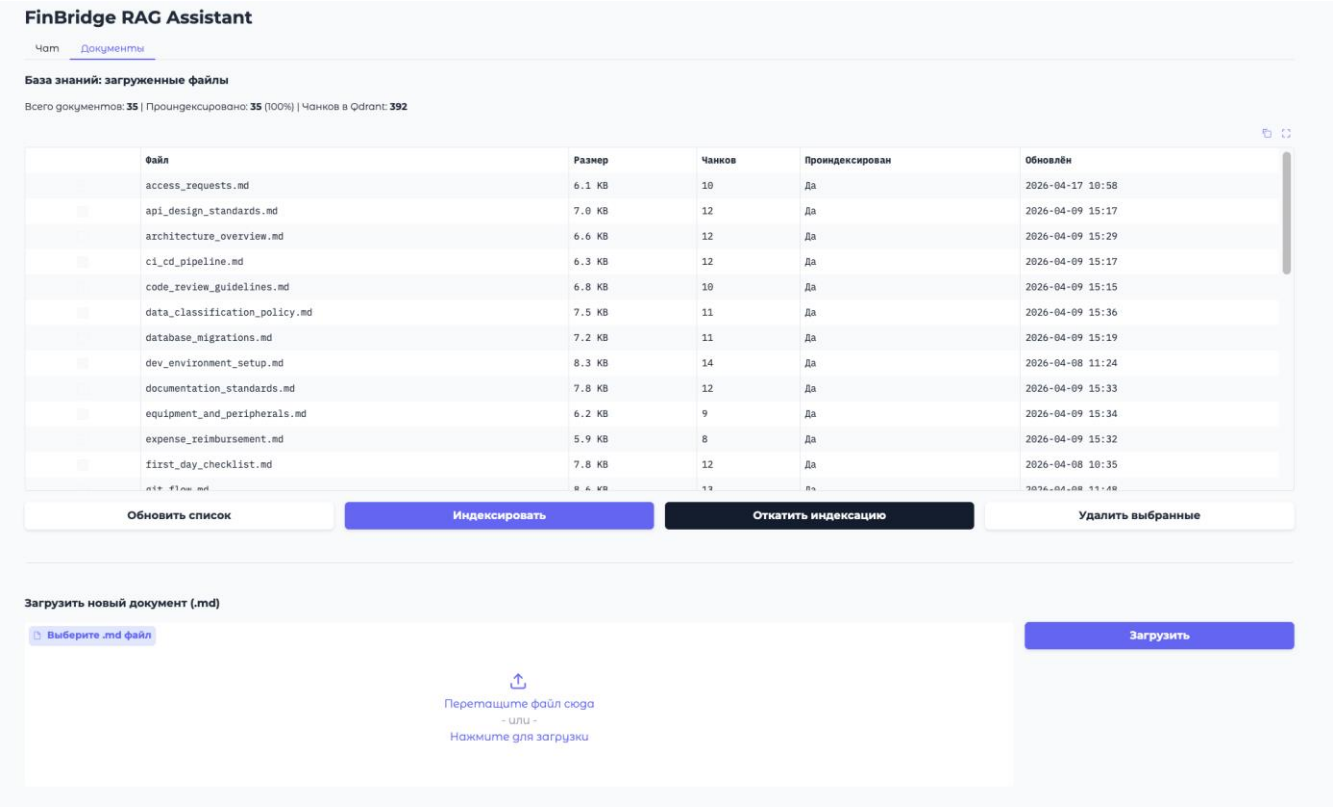


Рис. 5 - Административный интерфейс на Gradio

Глава 3. Разработка RAG-pipeline

3.1. Архитектура пайплайна загрузки и индексации документов

RAG-пайплайн системы организован как двухфазный процесс с чётким разделением ответственности. Фаза индексации инициируется явно через административный API и обрабатывает документы из директории finbridge/docs/. Фаза поиска вызывается автоматически на каждый пользовательский запрос, классифицированный как обращение к базе знаний.

Вся логика индексирующей фазы инкапсулирована в классе DocsDirectoryIngestion (модуль service/RAG/docs_service.py). Класс реализует паттерн статических методов-сервисов и предоставляет интерфейс: docs_load — загрузка и индексация; erase_docs — удаление из Qdrant и файловой системы; get_documents_info — статистика индексации; rm_points_by_source — адресное удаление векторов конкретного документа.

Пайплайн обработки одного документа проходит четыре стадии: загрузка текста из файла .md через TextLoader (LangChain); разбиение на чанки через RecursiveCharacterTextSplitter; вычисление эмбеддинга каждого чанка моделью BAAI/bge-small-en; пакетная загрузка чанков в Qdrant через QdrantVectorStore.add_documents. Каждый чанк сохраняется с payload, содержащим полный путь к файлу-источнику, что позволяет при удалении документа точечно удалять только его векторы по фильтру metadata.source.

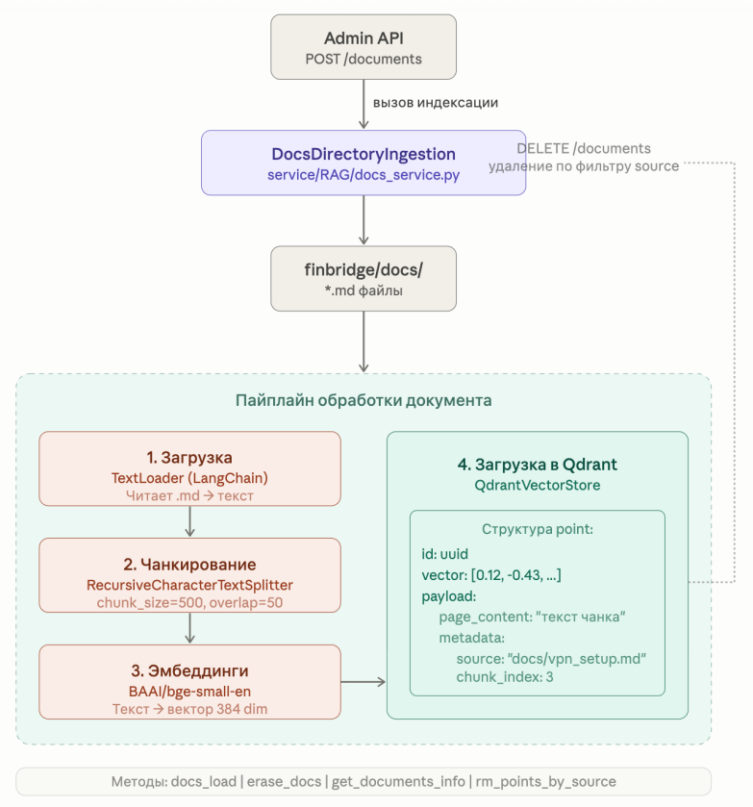


Рис. 6 - Схема работы Ingestion PipeLine

3.2. Стратегии сегментации текста: сравнительный анализ

Качество RAG-системы в значительной мере определяется стратегией сегментации документов. Размер чанка и степень перекрытия оказывают прямое влияние на точность семантического поиска: слишком маленький чанк теряет контекст, слишком большой «размывает» семантику и может выходить за пределы контекстного окна модели эмбеддингов.

Фиксированный размер (Fixed-size chunking). Документ разбивается на фрагменты равной длины с заданным перекрытием. Недостатки: границы разреза не учитывают смысловую структуру, чанк может оборваться посередине предложения.

Рекурсивное разбиение по разделителям (Recursive splitting). Разбиение ведётся рекурсивно с приоритетным перечнем: двойной перенос строки → одинарный → точка → пробел → символ. Алгоритм предпочитает наиболее крупный разделитель и только при превышении целевого размера опускается к следующему. Максимально сохраняет смысловые блоки при соблюдении ограничений на длину.

Семантическое разбиение (Semantic chunking). Предложения объединяются в чанки на основе попарного семантического сходства их эмбеддингов. Обеспечивает наилучшее смысловое единство, однако требует дополнительного вычисления эмбеддингов на этапе индексации и порождает фрагменты сильно неравномерной длины.

Разбиение по структуре Markdown (Structure-aware chunking). Для форматов с явной иерархической структурой разбиение ведётся по заголовкам разделов. Даёт максимально релевантные фрагменты, но может порождать чанки со значительным разбросом в размере.

Таблица 1 — Сравнение стратегий чанкирования

Стратегия	**Сохранение смысла**	**Равномерность размера**	**Сложность реализации**	**Учёт структуры**
Фиксированный размер	Низкое	Высокая	Низкая	Нет
Рекурсивное разбиение	Среднее	Среднее	Низкая	Частично
Семантическое разбиение	Высокое	Низкая	Высокая	Нет
По заголовкам Markdown	Высокое	Низкая	Средняя	Да

3.3. Реализованная схема чанкирования

В настоящей работе применяется стратегия рекурсивного разбиения через RecursiveCharacterTextSplitter с параметрами: chunk_size=512, chunk_overlap=64, separators=["\n", " ", "."].

Во-первых, документальная база сформирована в Markdown, хорошо совместимом с разбиением по переносам строк: каждый абзац отделяется пустой строкой и представляет завершённую мысль. Рекурсивный алгоритм предпочтёт разбиение на границе абзаца и опустится к следующему разделителю только при необходимости.

Во-вторых, chunk_size=512 согласован с ограничением модели эмбеддингов BAAI/bge-small-en: рекомендованная максимальная длина входной последовательности составляет 512 токенов, превышение которой приводит к усечению и деградации качества вектора.

В-третьих, перекрытие в 64 символа обеспечивает передачу граничного контекста между соседними чанками: предложение, оборванное на границе, разделяется обоими соседними фрагментами, повышая вероятность поиска нужного чанка при вопросе, охватывающем обе стороны границы.

3.4. Построение векторного хранилища

Управление векторным хранилищем централизовано в синглтоне QdrantService (service/vectorstore/client.py). Паттерн синглтона применён намеренно: инициализация клиента Qdrant и загрузка модели эмбеддингов занимают несколько секунд и должны выполняться один раз за время жизни приложения.

При первом обращении к методу get() класс создаёт клиент QdrantClient с параметрами из конфигурации, загружает HuggingFaceEmbeddings, и при отсутствии коллекции создаёт её с метрикой COSINE и размерностью, определяемой автоматически из модели. Метод get_qdrant_vector_store() возвращает объект QdrantVectorStore, обёрнутый @lru_cache(maxsize=256), что исключает повторную инициализацию при высокочастотных запросах.

Структура точки в Qdrant: поле vector хранит вектор размерностью 384; payload содержит page_content (текст чанка) и metadata.source (абсолютный путь к файлу-источнику). По полю source выполняется удаление чанков документа при его исключении из базы знаний.

3.5. Алгоритм семантического поиска

При поступлении запроса метод AgenticToolset.retrieve_context преобразует вопрос моделью BAAI/bge-small-en в вектор размерностью 384 — запросный вектор. QdrantVectorStore.similarity_search(question, k=10) вычисляет косинусное сходство между

запросным вектором и всеми точками коллекции, возвращая 10 ближайших документов в порядке убывания сходства.

Найденные чанки сериализуются в строку формата «Source: {файл} / Content: {текст}» и передаются агенту как контекст. Значение k=10 выбрано с расчётом на полноту охвата: вопрос может затрагивать несколько документов одновременно, а контекстное окно phi3:mini (~8000 токенов) с запасом вмещает 10 фрагментов по 512 символов вместе с системным промптом. Косинусная метрика инвариантна к длине векторов и корректно сопоставляет короткий вопрос с длинным чанком при семантическом перекрытии.

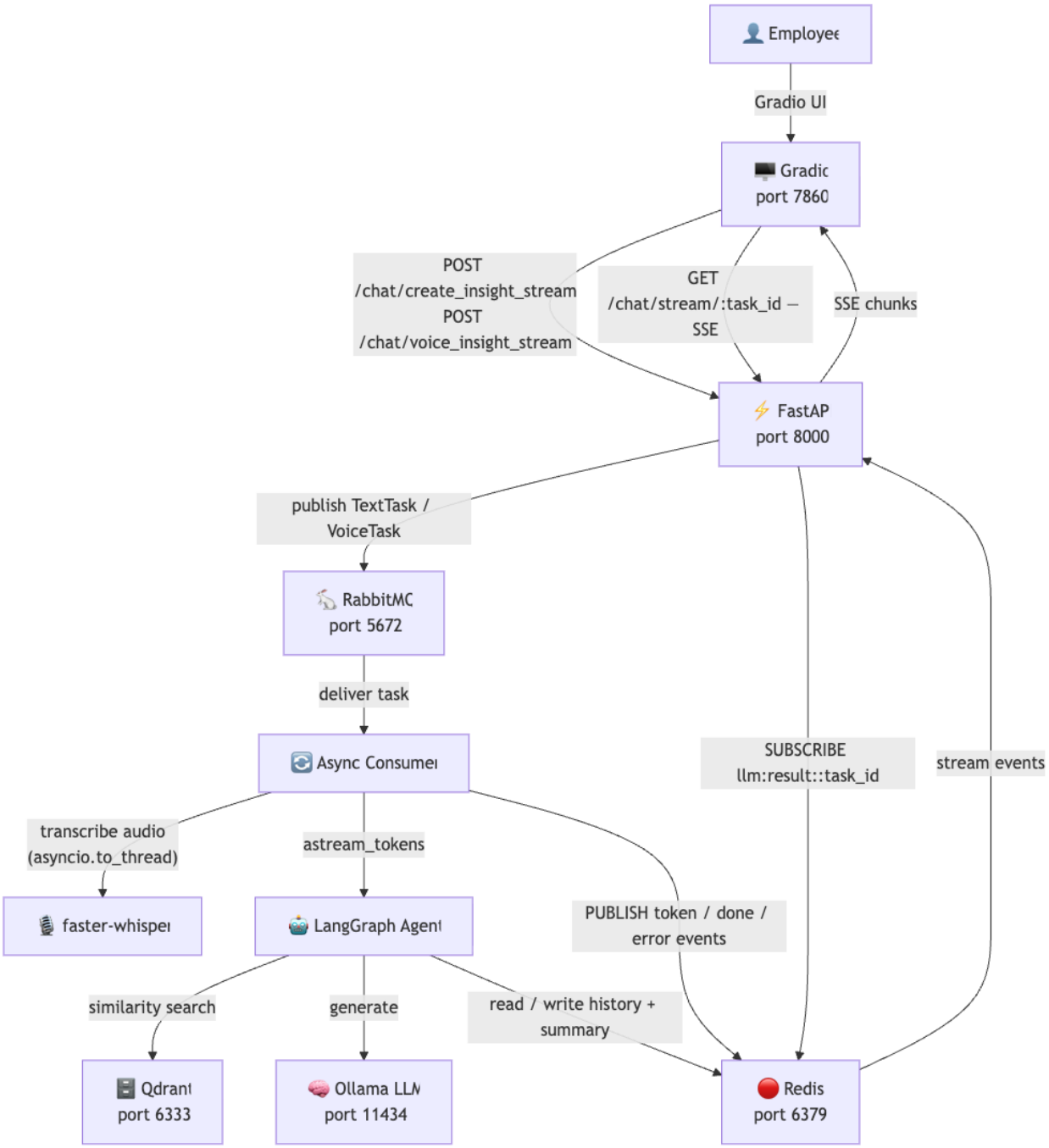


Рис. 7 - Общая архитектура приложения

Глава 4. Интеллектуальный агент на основе LangGraph

4.1. Архитектура агента

Интеллектуальный агент системы реализован в модуле service/bot/agent.py в виде синглтона RAGAgent. Агент представляет собой ориентированный граф состояний, построенный с помощью StateGraph из LangGraph, и экспортирует единственный публичный интерфейс — асинхронный генератор astream_tokens(question, session_id), вызываемый обработчиком задач RabbitMQ.

Выбор LangGraph обусловлен тремя архитектурными требованиями:

- Во-первых, поведение агента должно ветвиться в зависимости от намерения запроса, и LangGraph делает этот граф явным и наглядным.
- Во-вторых, при стриминге токенов необходимо знать, в каком именно узле был сгенерирован токен — astream_events позволяет фильтровать события по имени узла.
- В-третьих, StateGraph упрощает добавление новых узлов без рефакторинга существующих.

4.2. Описание графа состояний

Состояние агента описывается Pydantic-моделью State со следующими полями:

- history: list[BaseMessage] — последние полные 6 сообщений из диалога;
- summary: str — прогрессивное резюме всей сессии;
- question: str — текущий вопрос пользователя;
- intent: str — результат классификации (knowledge_base | small_talk);
- answer: str — финальный ответ агента;
- docs: List[Document] — список документов-источников.

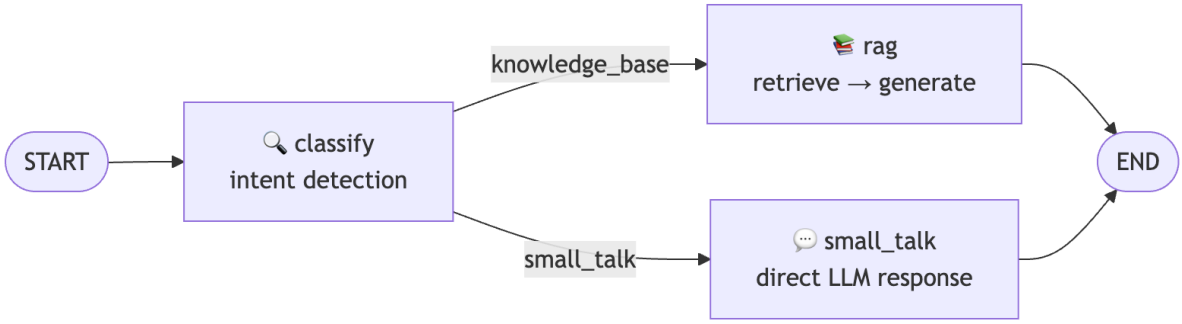


Рис. 8 - Топология графа

4.3. Узел классификации намерений

Узел classify_node вызывается первым для каждого запроса и определяет маршрут обработки. Классификация реализована через AgenticToolset.classify_intent: формируется

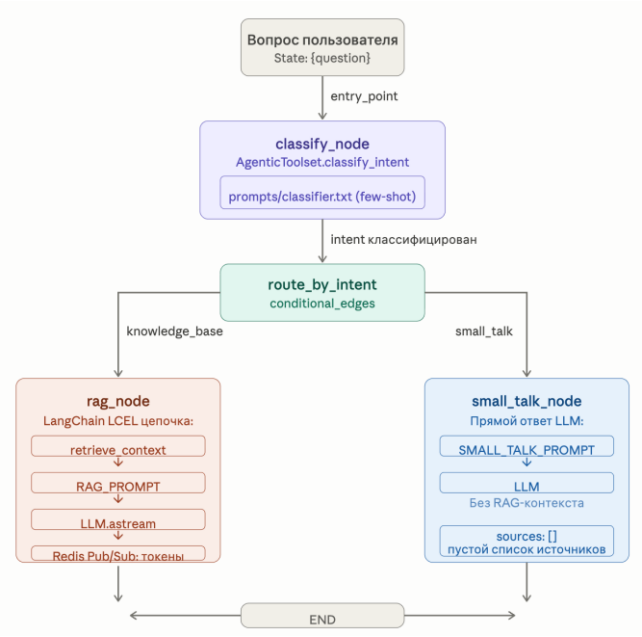
промпт-шаблон из файла prompts/classifier.txt, в который подставляется текущий вопрос; LLM возвращает одно из двух значений — knowledge_base или small_talk.

Промпт классификатора построен по технике few-shot: несколько примеров вопросов каждого класса обеспечивают стабильную классификацию даже с небольшой моделью. Вывод обрабатывается StrOutputParser; при неопределённом выводе цепочка направляет запрос в RAG-узел по умолчанию. Данный подход выбран вместо tool-calling: явная одношаговая классификация через промпт даёт результат с минимальной задержкой.

4.4. RAG-узел: ответы по базе знаний

Узел rag_node обрабатывает запросы к корпоративной базе знаний. Внутри узла выполняется цепочка LangChain Expression Language:

```
retrieve_context | RAG_PROMPT | LLM | StrOutputParser
```



Функция retrieve_context вызывает QdrantVectorStore.similarity_search(question, k=10) и возвращает строку сериализованных фрагментов и список Document. Промпт RAG_PROMPT включает системную инструкцию, историю диалога, резюме сессии, вопрос и найденный контекст. Генерация выполняется через потоковый интерфейс LLM (astream). Метод astream_tokens подписывается на события on_chat_model_stream через graph.astream_events и публикует каждый токен в Redis. Дополнительно обрабатывается on_chain_end в узле rag — из него извлекается список источников для финального события done.

4.5. Узел обработки обычный запросов

Узел small_talk_node отвечает на запросы, не требующие обращения к базе знаний: приветствия, вопросы о возможностях системы, общие темы. Языковая модель отвечает напрямую через цепочку SMALL_TALK_PROMPT | LLM. Промпт информирует модель о её роли (корпоративный ассистент FinBridge) и предписывает сообщать о специализации при выходе за рамки. Узел возвращает пустой список источников.

4.6. Управление диалоговой памятью

Персистентность памяти реализована в классе RedisHistoryStore (service/ShortTermMemory/redis_storage.py) — статическом адаптере над RedisChatMessageHistory из langchain-community. При каждом вызове astream_tokens выполняется: извлечение последних 6 сообщений и накопленного резюме из Redis; добавление вопроса пользователя в историю; передача всех трёх элементов в начальное состояние графа; после генерации — добавление ответа агента в историю и обновление резюме через цепочку SUMMARIZATION_PROMPT | LLM | StrOutputParser.

Прогрессивная суммаризация предотвращает бесконечный рост истории и переполнение контекстного окна LLM: размер входного контекста остаётся предсказуемо ограниченным (6 сообщений + одно резюме). Время жизни ключей Redis (TTL) установлено равным 1 часу, что соответствует типичной рабочей сессии нового сотрудника.

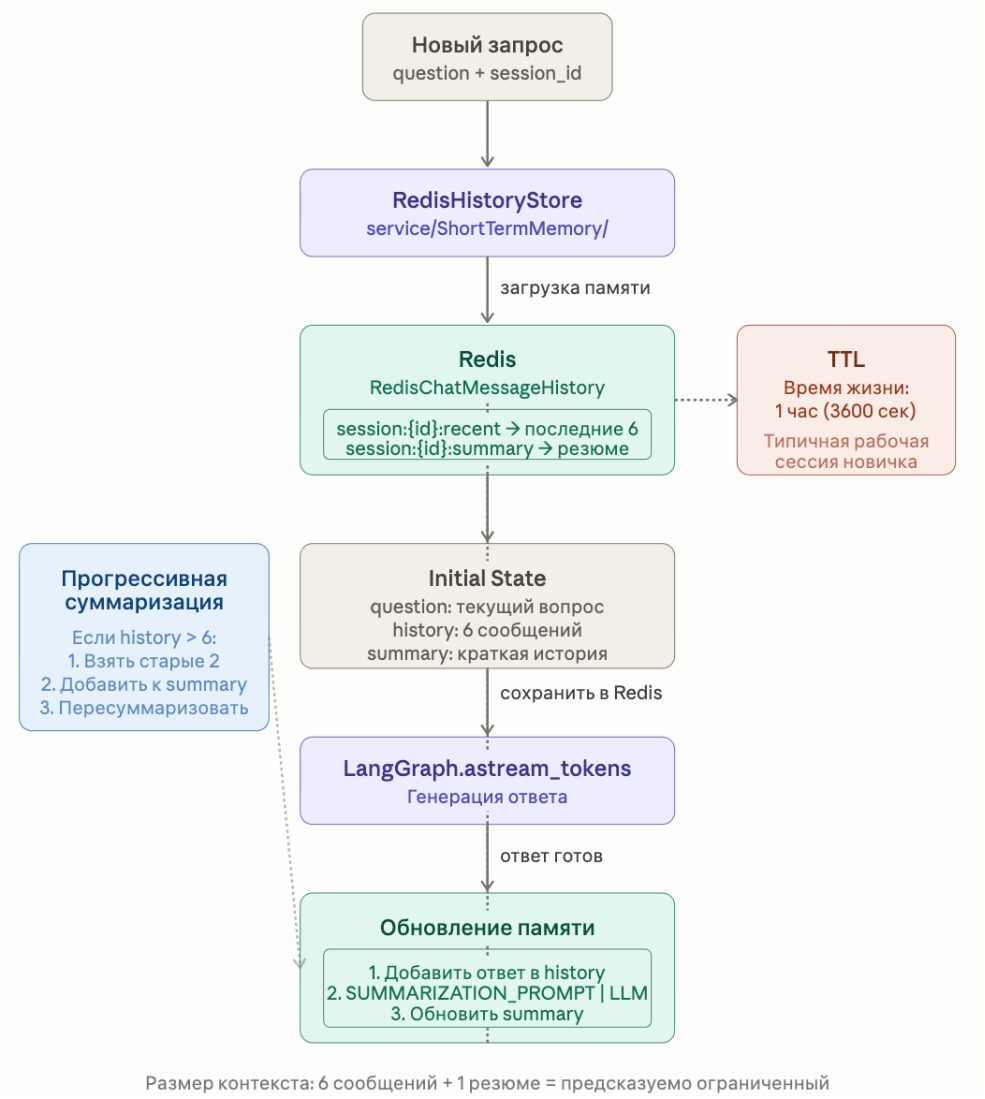


Рис. 10 - Схема устройства памяти

Глава 5. Асинхронная обработка задач с использованием RabbitMQ

5.1. Обоснование применения брокера сообщений

В системах с длительными вычислительными операциями наивная синхронная обработка HTTP-запросов порождает две критические проблемы. Проблема задержки ответа: HTTP-клиент блокируется в ожидании завершения операции (генерация LLM и транскрибация занимают от 5 до 60 секунд), тогда как стандартный таймаут HTTP-клиентов составляет 30–60 секунд. Проблема блокирования сервера: долгоживущие соединения ограничивают параллелизм.

Паттерн «задача — очередь — воркер» (Task Queue Pattern) разрывает эту связь: HTTP-эндпоинт мгновенно регистрирует запрос, возвращает task_id и завершает соединение; тяжёлые вычисления выполняются фоновым воркером. Клиент получает результат через отдельный SSE-поток по мере готовности токенов.

Таблица 2 — Сравнение инструментов для очереди задач

Критерий	RabbitMQ +aio-pika	Celery + Redis	Kafka	Redis Streams
Asyncio (нативно)	Да	Нет	Частично	Да
Семантика ACK/NACK	Да	Да	Да	Да
Гибкая маршрутизация	Да (exchange)	Ограничена	По разделам	Ограничена
Dead-letter queue	Встроена	Через конфиг	Да	Нет
Веб-интерфейс	Встроен (Порт: 15672)	Flower	Kafka UI	Redis Insight
Сложность deploy	Низкая	Средняя	Высокая	Низкая

RabbitMQ с aio-pika признан оптимальным: нативная asyncio-интеграция, гибкая маршрутизация через exchange, встроенная dead-letter queue и удобный Management UI при минимальной сложности развёртывания.

5.2. Топология обмена сообщениями

Брокерская топология реализована как один DIRECT exchange с именем finbridge. К обмену привязаны две очереди: finbridge.text (ключ task.text) — для текстовых задач; finbridge.voice (ключ task.voice) — для голосовых задач.

Разделение очередей по типам обосновано различием в вычислительной интенсивности: голосовые задачи требуют предварительной транскрибации (2–10 секунд), тогда как текстовые переходят сразу к генерации. Выделенные очереди позволяют в будущем настраивать разное число воркеров для каждого типа независимо. Сообщения публикуются с DeliveryMode.PERSISTENT, что гарантирует сохранность очереди на диске при перезапуске брокера.

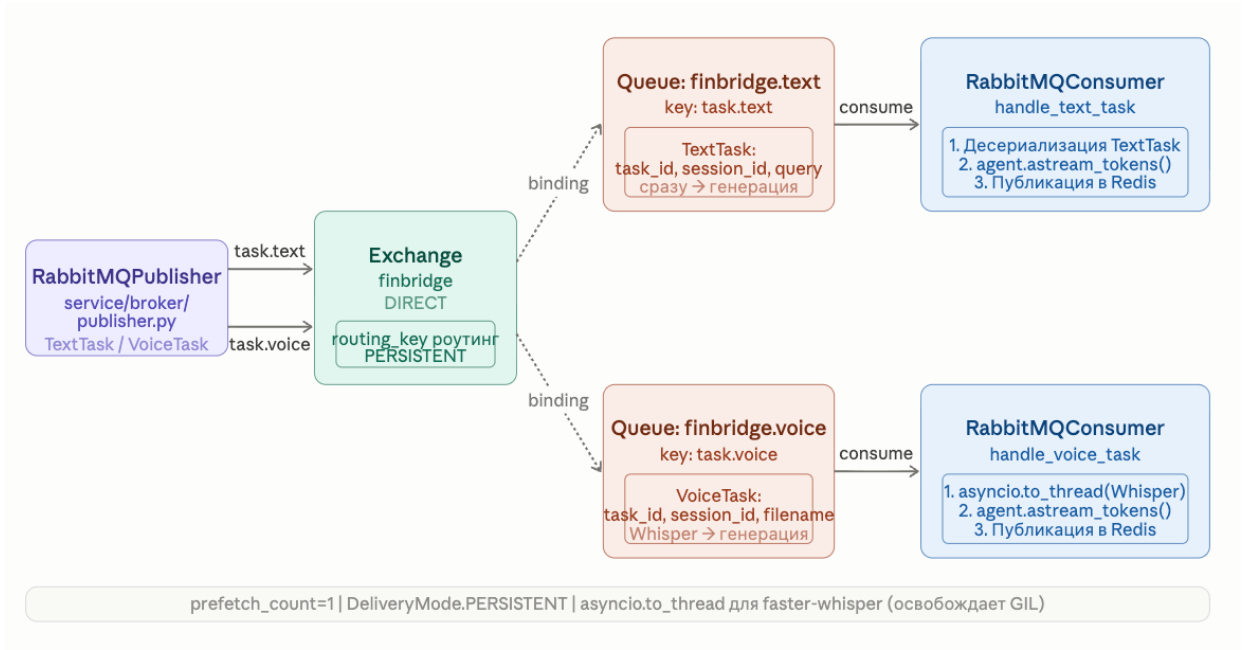


Рис. 11 - Устройство очередей RabbitMQ

5.3. Публикатор задач

Публикатор реализован в RabbitMQPublisher (service/broker/publisher.py). Метод connect() устанавливает asyncio-соединение через aio_pika.connect_robust, декларирует exchange и очереди.

Задачи описываются Pydantic-моделями:

- TextTask содержит task_id (UUID4), session_id, query и t_type=TEXT;
- VoiceTask — task_id, session_id, filename (путь к временному аудиофайлу) и t_type=VOICE.

Временный аудиофайл создаётся в HTTP-обработчике FastAPI через tempfile.NamedTemporaryFile, а путь к нему передаётся воркеру через VoiceTask.filename. Воркер самостоятельно читает байты файла после получения задачи. Такой подход избавляет от необходимости сериализовывать бинарные данные в JSON или хранить их в Redis.

5.4. Потребитель сообщений и обработчики задач

Потребитель RabbitMQConsumer (service/broker/consumer.py) запускается как долгоживущая asyncio-задача в lifespan FastAPI. Параметр prefetch_count=1 ограничивает число одновременно обрабатываемых сообщений одним — это защищает воркер от перегрузки при очереди тяжёлых задач. Обработка ведётся внутри msg.process(), который автоматически отправляет ACK при успехе и NACK при исключении.

Обработчик handle_text_task десериализует тело в TextTask, вызывает генератор agent.astream_tokens(query, session_id) и для каждого события публикует токен в Redis. При исключении публикуется событие {type: error}.

Обработчик handle_voice_task добавляет шаг транскрибации: читает байты аудиофайла, запускает WhisperTranscriber.transcribe через asyncio.to_thread, публикует событие {type: transcript}, затем вызывает agent.astream_tokens. Применение asyncio.to_thread критично: синхронный вызов faster-whisper блокирует GIL Python и полностью останавливает event loop; перенос в поток освобождает цикл для SSE-соединений.

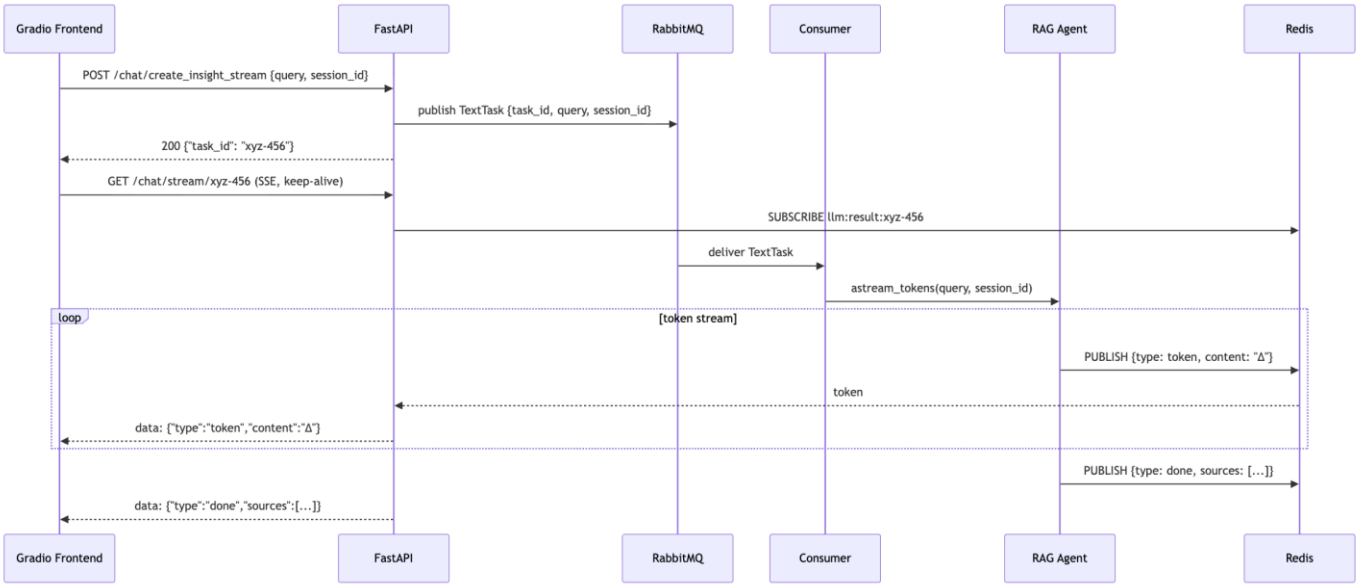


Рис. 12 - Поток текстового сообщения

Глава 6. Потоковая передача ответов: Redis Pub | Sub и SSE

6.1. Архитектурное решение потоковой передачи

Доставка токенов организована по трёхзвенной схеме: воркер RabbitMQ → Redis Pub/Sub → SSE-эндпоинт FastAPI → браузер. Данная архитектура обусловлена изолированностью компонентов: воркер не имеет прямой ссылки на HTTP-соединение конкретного клиента. Redis выступает надёжным посредником, изолирующим события по идентификатору задачи.

Для каждого запроса создаётся уникальный канал Redis с ключом llm:result:{task_id}. Клиент немедленно открывает SSE-соединение к /chat/stream/{task_id}, который подписывается на тот же канал. Событийная модель включает четыре типа: token (очередной токен текста), transcript (расшифровка голосового запроса), done (сигнал завершения с источниками), error (сообщение об ошибке).

6.2. Реализация Redis Pub/Sub для доставки токенов

Класс RedisResultStore (service/broker/result_store.py) инкапсулирует логику публикации и подписки. Метод publish_event(task_id, event: dict) сериализует словарь через json.dumps и публикует строку в канал llm:result:{task_id}. Метод stream_events(task_id) является асинхронным генератором: создаёт новое независимое соединение Redis, подписывается на канал, итерируется через pubsub.listen() и при получении события done/error завершает работу.

Создание отдельного соединения на каждый SSE-запрос принципиально: объект pubsub занимает соединение и несовместим с разделяемым пулом. Поскольку каждая задача имеет уникальный task_id, каналы полностью изолированы: публикация токенов для одной задачи не влияет на подписки других, что обеспечивает корректную обработку при любом числе одновременных пользователей.

6.3. Server-Sent Events в FastAPI

Server-Sent Events (SSE) — стандартный протокол HTTP для инициирования передачи данных сервером в рамках долгоживущего GET-запроса. В отличие от WebSocket, SSE является однонаправленным и опирается на стандартный HTTP, что упрощает работу через прокси. Формат события: data: {payload}\n\n.

В FastAPI SSE-эндпоинт GET /chat/stream/{task_id} реализован через StreamingResponse. Генератор оборачивает каждую строку из stream_events в формат SSE. Заголовок Cache-Control: no-cache запрещает кэширование промежуточными прокси. Заголовок X-Accel-Buffering: no отключает буферизацию nginx: без него nginx удерживал бы данные до заполнения буфера (4–8 КБ), нивелируя эффект стриминга.

На стороне клиента (Gradio) обработка SSE реализована через httpx с response.aiter_lines(). При token — символы дописываются к ответу ассистента; при transcript — обновляется отображение вопроса; при done — добавляются источники и соединение закрывается.

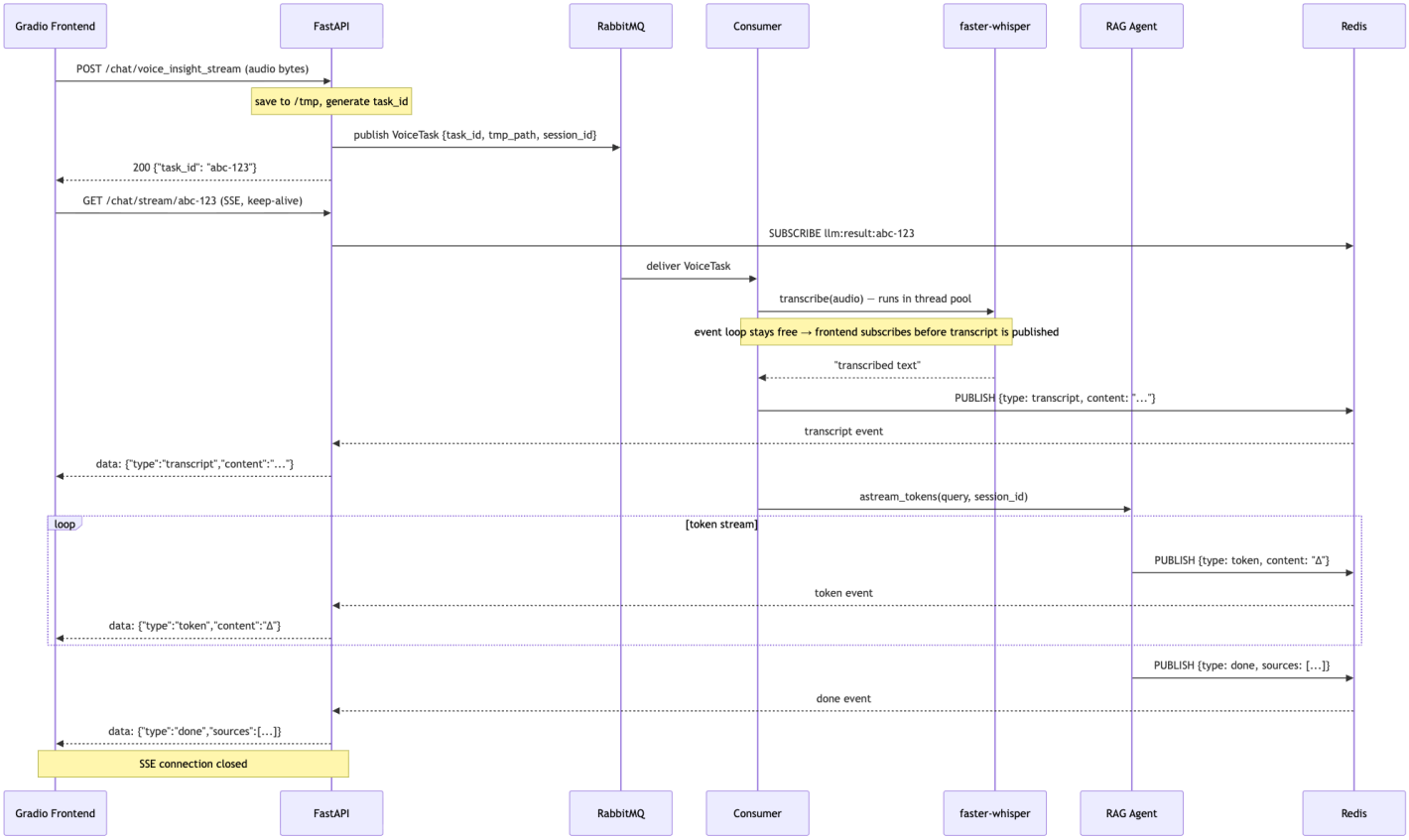


Рис. 13 - Поток голосового сообщения

Глава 7. Мультимодальный ввод: Обработка голосовых запросов

7.1. Обоснование выбора ASR-движка

Для реализации автоматического распознавания речи (ASR) было проведено сравнение четырёх альтернатив.

- OpenAI Whisper (оригинал) — высочайшее качество, но реализация на PyTorch с float32 работает на CPU в 3–5 раз дольше.
- Google Cloud Speech-to-Text — передача аудио на внешние серверы противоречит требованию конфиденциальности + платная модель.
- Vosk — высокая скорость, но качество (WER) заметно уступает Whisper на русском языке и корпоративной лексике.
- Faster-whisper — ускоренная реализация Whisper через CTranslate2 с int8: та же точность при 2–4-кратном ускорении на CPU, полностью локальная, открытый код.

Таблица 3 — Сравнение ASR-движков

Критерий	OpenAI Whisper	Faster-Whisper	Google STT	Vosk
Локальность	Да	Да	Нет	Да
Качество	Высокое	Высокое	Высокое	Среднее
Скорость (CPU, base)	Низкая	Средняя	—	Высокая
Открытый код	Да	Да	Нет	Да
Автоопр. языка	Да	Да	Да	Нет
Русский язык	Да	Да	Да	Да
Бесплатность	Да	Да	Частично	Да

Faster-whisper признан оптимальным выбором: сочетает высокое качество оригинального Whisper, требование локальности, открытость кода и приемлемую скорость на CPU.

7.2. Реализация модуля транскрибации

Модуль реализован в классе WhisperTranscriber (service/whisper/transcriber.py) по паттерну синглтона с ленивой инициализацией: загрузка весов занимает 1–5 секунд и выполняется один раз при первом обращении через _get_model(). При инициализации создаётся WhisperModel(WHISPER_MODEL, device="cpu", compute_type="int8"): квантизация int8 ускоряет вывод при снижении качества менее чем на 0,5–1%.

Метод transcribe(audio_bytes, filename) определяет расширение файла для корректного формата, записывает байты во временный файл, вызывает model.transcribe(path, beam_size=5) и объединяет сегменты в строку. Параметр beam_size=5 — стандартный компромисс точность/скорость для модели base. Класс логирует обнаруженный язык и вероятность его определения.

7.3. Интеграция распознавания речи в общий пайплайн

Транскрибация интегрируется в handle_voice_task через asyncio.to_thread: синхронный CPU-bound вызов faster-whisper выполняется в пуле потоков (ThreadPoolExecutor), освобождая event loop для параллельной обработки SSE-соединений и входящих запросов.

Именно это позволяет клиентскому браузеру успеть установить SSE-соединение ещё до завершения транскрибации: расшифровка доставляется как первое SSE-событие transcript до начала генерации ответа. После получения транскрипции поток обработки полностью сливается с текстовым: агент вызывается с распознанным текстом через те же classify → [rag | small_talk] → стриминг токенов.

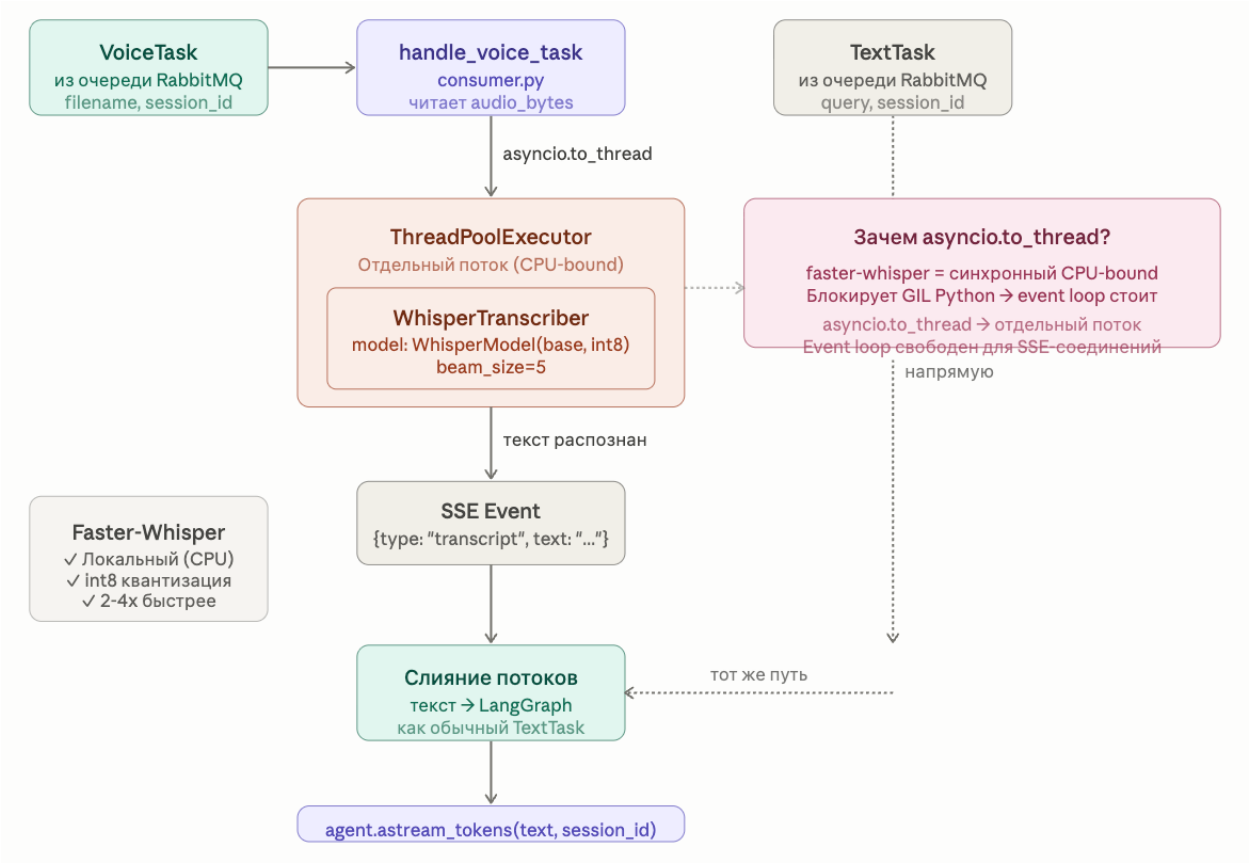


Рис. 14 - Схема транскрибации аудио

Глава 8. Пользовательский интерфейс и управление базой знаний

8.1. Интерфейс чата

Пользовательский интерфейс реализован на Gradio и структурирован в классе ChatTab (client/tabs/chat_tab.py). Разделение кода по файлам — один класс на одну вкладку — обеспечивает независимость компонентов и упрощает их поддержку.

Компоновка вкладки: виджет gr.Chatbot (высота 600px) для истории диалога; gr.Textbox с кнопкой «Отправить»; gr.Audio(sources=["microphone"]) для голосового ввода с автоостановкой; кнопка «Очистить»; скрытый gr.State для session_id. При первой загрузке страницы _ensure_session вызывает POST /chat/sessions и сохраняет UUID в State; идентификатор передаётся в заголовке X-Session-Id.

Обработчик send_message работает как асинхронный Gradio-генератор, создавая иллюзию «живого» набора текста. Генератор stream_sse открывает httpx.AsyncClient с timeout=None к /chat/stream/{task_id} и обрабатывает события: transcript → обновляет вопрос; token → дописывает к ответу ассистента; done → добавляет ссылки на источники; error → выводит текст ошибки.

8.2. Модуль управления документами

Административный интерфейс реализован в DocTabUI (client/tabs/documents_tab.py). Центральным элементом является интерактивная таблица gr.DataFrame с колонками: чекбокс выбора, имя файла, размер (КБ), число чанков, статус индексации, дата изменения. Параметр interactive=True позволяет отмечать строки без дополнительного кода.

Кнопки «Индексировать» / «Откатить индексацию» вызывают indexation_selected: при пустом выборе операция применяется ко всем документам. Кнопка «Удалить выбранные» вызывает DELETE /admin/documents/delete и отображает статистику удалённых чанков. Загрузка файлов — gr.File(file_types=[".md"]) + POST /admin/documents/upload с проверкой расширения.

8.3. REST API администрирования

Административный API описан в service/api/Documents/admin_router.py (префикс /admin). Все данные описаны Pydantic-моделями — автоматическая документация Swagger UI.

GET /admin/documents возвращает DocumentsListResponse со списком DocumentEntry (имя, размер, число чанков, флаг индексации, дата) и DocumentsSummary со сводной статистикой.

POST /admin/documents/upload принимает .md-файл через UploadFile, проверяет расширение и сохраняет в DOCS_DIR. Статус-код ответа 201 Created.

POST /admin/documents/reindex принимает ReindexRequest с filenames и action ("Index" | "Unindex"). При Index вызывает полный пайплайн чанкирования; при Unindex — только удаление векторов через rm_points_by_source без удаления файлов. При null filenames операция применяется ко всем документам.

DELETE /admin/documents/delete принимает список RequiredDocsInteraction и удаляет векторы из Qdrant и файлы с диска. Возвращает DeletedChunkDoc с детальной статистикой для каждого документа.

Глава 9. Конфигурирование и развёртывание системы

9.1. Управление конфигурацией через Pydantic Settings

Все параметры системы централизованы в классе Settings (config.py), наследующем BaseSettings из pydantic-settings. Подход обеспечивает типизированный доступ, автоматическую валидацию при запуске и загрузку из .env и переменных окружения. Единственный экземпляр settings = Settings() импортируется из config в остальных модулях, исключая дублирование чтения переменных.

Параметры сгруппированы по блокам:

- Ollama: AI_API_KEY, BASE_URL, MODEL — подключение к локальной LLM.
- Qdrant: QDRANT_HOST, QDRANT_PORT, QDRANT_COLLECTION_NAME, EMBEDDINGS_MODEL_NAME.
- Redis: REDIS_URL, REDIS_TTL — подключение и TTL ключей сессии.
- RabbitMQ: RABBITMQ_USER, RABBITMQ_PASS, RABBITMQ_HOST, RABBITMQ_PORT.
- Whisper: WHISPER_MODEL — размер модели ASR (tiny / base / small / medium / large).
- Gradio: API_URL, GRADIO_PORT — адрес API и порт фронтенда.

9.2. Жизненный цикл FastAPI-приложения

Запуск и остановка долгоживущих соединений управляются через контекстный менеджер lifespan FastAPI, гарантирующий инициализацию компонентов до обработки первого HTTP-запроса и корректное освобождение ресурсов при завершении.

Последовательность при старте:

1. Установка уровня журналирования INFO.
2. publisher.connect() — соединение с RabbitMQ, декларирование exchange и очередей.
3. result_store.connect() — инициализация Redis-клиента для публикации событий.
4. consumer.connect() — установка AMQP-соединения потребителя.
5. asyncio.create_task(consumer.start_consuming(...)) — запуск воркера как фоновой задачи.

При остановке вызываются close() для Publisher, ResultStore и Consumer, корректно закрывающие AMQP и Redis-соединения.

9.3. Контейнеризация с Docker Compose

Вся инфраструктура системы разворачивается через Docker Compose единой командой docker compose up -d. Файл docker-compose.yml описывает четыре сервиса.

- Qdrant (qdrant/qdrant:latest): порты 6333 (REST) и 6334 (gRPC), том qdrant_storage для персистентности индекса.
- Redis (redis:latest): порт 6379, том redis_data.
- RabbitMQ (rabbitmq:3-management): порты 5672 (AMQP) и 15672 (Management UI)
- Ollama (ollama/ollama): порт 11434, том ollama_models. После запуска необходимо однократно выполнить docker exec ollama ollama pull phi3:mini.

Само Python-приложение (FastAPI + Gradio) запускается вне Docker: при активной разработке горячая перезагрузка uvicorn --reload и отладчик работают удобнее в нативном окружении. Полный перечень переменных окружения приведён в таблице 4.

Таблица 4 — Переменные окружения системы

Переменная	Описание	Пример значения
MODEL	Имя LLM-модели в Ollama	deepseek-r1:1.5b
BASE_URL	URL Ollama API	http://localhost:11434
AI_API_KEY	Ключ API (для Ollama — произвольный)	ollama
QDRANT_HOST	Хост Qdrant	http://localhost
QDRANT_PORT	Порт Qdrant	6333
QDRANT_COLLECTION_NAME	Имя коллекции	finbridge_docs
REDIS_URL	URL подключения к Redis	redis://localhost:6379
REDIS_TTL	TTL ключей сессии (сек)	3600
RABBITMQ_USER	Логин RabbitMQ	guest
RABBITMQ_PASS	Пароль RabbitMQ	guest
RABBITMQ_HOST	Хост RabbitMQ	localhost
RABBITMQ_PORT	Порт AMQP	5672
EMBEDDINGS_MODEL_NAME	Модель эмбеддингов HuggingFace	BAAI/bge-small-en
WHISPER_MODEL	Размер модели Whisper	base
API_URL	Базовый URL FastAPI для Gradio	http://localhost:8000
GRADIO_PORT	Порт Gradio	7860

Заключение

В ходе выполнения выпускной квалификационной работы была спроектирована и реализована интеллектуальная система поддержки онбординга сотрудников на основе RAG-архитектуры с мультимодальным вводом. По каждой из поставленных задач достигнуты следующие результаты.

Задача 1. Разработка RAG-пайплайна. Реализован полный пайплайн загрузки, сегментации и индексации документов в векторную базу данных Qdrant. Проведён сравнительный анализ четырёх стратегий чанкирования; выбрано рекурсивное разбиение с параметрами chunk_size=512, chunk_overlap=64. Семантический поиск реализован на основе BAAI/bge-small-en и косинусной метрики Qdrant с возвратом 10 ближайших фрагментов.

Задача 2. Разработка интеллектуального агента. Реализован агент на основе StateGraph из LangGraph с тремя узлами: классификатором намерений, RAG-узлом и узлом светской беседы. Агент поддерживает персистентный контекст через хранение 6 последних сообщений и прогрессивного резюме в Redis. Стриминг токенов реализован через astream_events с фильтрацией по имени узла.

Задача 3. Асинхронная обработка задач. Внедрён брокер RabbitMQ с DIRECT exchange и двумя очередями — для текстовых и голосовых задач. HTTP API возвращает task_id немедленно; вычисления выполняются фоновым потребителем с prefetch_count=1. Модели задач описаны через Pydantic.

Задача 4. Потоковая доставка ответов. Реализован механизм передачи токенов по цепочке воркер → Redis Pub/Sub → SSE → браузер. Изолированный Redis-канал на задачу обеспечивает корректную маршрутизацию при любом числе параллельных пользователей. Отключение буферизации nginx гарантирует немедленную доставку каждого токена.

Задача 5. Мультимодальный ввод. Интегрирован модуль ASR на базе faster-whisper с квантизацией int8. Синхронный вызов транскрибации выполняется в потоке через asyncio.to_thread, освобождая event loop. Поддерживается автоопределение языка; транскрипция доставляется как первое SSE-событие до начала генерации ответа.

Задача 6. Пользовательский интерфейс и управление базой знаний. Разработан двухвкладочный веб-интерфейс на Gradio: вкладка чата с потоковым отображением ответов и поддержкой голосового ввода, административная вкладка с таблицей документов и операциями индексации/удаления. REST API реализован на FastAPI с Pydantic-моделями.

Система в полной мере соответствует всем сформулированным функциональным и нефункциональным требованиям. Все инфраструктурные компоненты развёртываются единой командой docker compose up -d; приложение запускается без зависимостей от внешних облачных сервисов.

Практическая ценность работы состоит в том, что реализованная система пригодна к адаптации для реальной корпоративной среды: замена документов на реальные сводится к загрузке .md-файлов через административный интерфейс без изменения кода. Архитектура RabbitMQ + Redis + LangGraph + Qdrant обеспечивает горизонтальную масштабируемость: запуск дополнительных воркеров-потребителей увеличивает пропускную способность без изменений в остальных компонентах. Языковая модель может быть заменена на более мощную через одну переменную окружения MODEL.

Направлениями дальнейшего развития системы могут стать: интеграция с корпоративным мессенджером (Telegram Bot API или Slack API); реализация гибридного поиска (комбинация векторного и полнотекстового BM25 для повышения точности); сравнительное тестирование различных языковых моделей на реальных пользовательских запросах.

Список литературы

1. Lewis P., Perez E., Piktus A. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // Advances in Neural Information Processing Systems. — 2020. — Vol. 33. — P. 9459–9474.

2. Brown T., Mann B., Ryder N. et al. Language Models are Few-Shot Learners // Advances in Neural Information Processing Systems. — 2020. — Vol. 33. — P. 1877–1901.

3. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. — 2019. — P. 3982–3992.

4. Radford A., Kim J. W., Xu T. et al. Robust Speech Recognition via Large-Scale Weak Supervision // International Conference on Machine Learning. — 2023.

5. Guu K., Lee K., Tung Z. et al. REALM: Retrieval-Augmented Language Model Pre-Training // International Conference on Machine Learning. — 2020. — P. 3929–3938.

6. Xiao S., Liu Z., Zhang P. et al. C-Pack: Packaged Resources To Advance General Chinese Embedding [Электронный ресурс]. — Режим доступа: https://arxiv.org/abs/2309.07597 (дата обращения: 05.05.2026).

7. Karpukhin V., Oguz B., Min S. et al. Dense Passage Retrieval for Open-Domain Question Answering // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. — 2020. — P. 6769–6781.

8. Izacard G., Grave E. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering // Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics. — 2021. — P. 874–880.

9. Mao Y., He P., Liu X. et al. Generation-Augmented Retrieval for Open-Domain Question Answering // Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics. — 2021. — P. 4089–4100.

10. Khattab O., Zaharia M. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT // Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. — 2020. — P. 39–48.

11. Johnson J., Douze M., Jégou H. Billion-scale similarity search with GPUs // IEEE Transactions on Big Data. — 2021. — Vol. 7. — No. 3. — P. 535–547.

12. Vaswani A., Shazeer N., Parmar N. et al. Attention is All You Need // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 5998–6008.

13. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // Proceedings of NAACL-HLT. — 2019. — P. 4171–4186.

14. Malkov Y. A., Yashunin D. A. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2020. — Vol. 42. — No. 4. — P. 824–836.

15. Vingelmann P., Fitzek F. H. P. CUDA, Release: 10.2.89 [Электронный ресурс]. — Режим доступа: https://developer.nvidia.com/cuda-toolkit (дата обращения: 05.05.2026).

16. Chase H. LangChain: Building applications with LLMs through composability [Электронный ресурс]. — Режим доступа: https://github.com/langchain-ai/langchain (дата обращения: 05.05.2026).

17. Vingelmann S. Faster Whisper: CTranslate2 implementation of OpenAI's Whisper model [Электронный ресурс]. — Режим доступа: https://github.com/SYSTRAN/faster-whisper (дата обращения: 05.05.2026).

18. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. — O'Reilly Media, 2017. — 616 p.

19. Richardson C., Smith F. Microservices Patterns: With Examples in Java. — Manning Publications, 2018. — 520 p.

20. Hohpe G., Woolf B. Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. — Addison-Wesley Professional, 2003. — 736 p.